

COSC 4367 — Project 1: The Alignment Engine/Edit Distance Experiments

- **Checkpoint due:** Friday September 25th, 11:59pm — design document §1–§3, a working core algorithm, and instrumentation wired in with `check()` passing
- **Final submission due:** Thursday October 8th, 11:59pm
- **Languages:** Java or Python. Any editor or IDE. If your solution needs packages beyond the standard library, bundle them or document exact install steps. As a reminder, code that fails to start will receive a 0.
- **Summary:** Create an implementation of the Edit Distance problem in three forms - Naive, Memoization, and Tabulation. Instrument your solution with our Counters file. Create a small test case for hand verification, and perform an empirical study with larger strings of specific alphabets, and analyze both raw operation counts and run time performance.

1. Part A — The core engine (required)

Implement **edit distance (Levenshtein distance)** between two strings: the minimum-cost sequence of single-character insertions, deletions, and substitutions that transforms string **A** into string **B**. Note that translating string **B** into string **A** may give different results, especially with varied weights.

A1. Naive recursive implementation

A2. Top-down memoized implementation

A3. Bottom-up tabulated implementation

A4. Solution reconstruction

Return not just the distance but an actual optimal edit script *and* a printed alignment. For **kitten** → **sitting** under unit costs:

```
None
k i t t e n -
s i t t i n g
S . . . S . I      (S = substitute, I = insert, D = delete, . =
match)
```

As you consider how to approach the solution, think of what must happen in addition to recognizing the subsequences (code we looked at on September 17th).

On ties: There will be strings for which multiple edit paths come to the same overall cost. It is your decision on which of those paths to follow. There are plentiful ways to break ties (first/last discovered, bias towards operation X, ...) Document your tie-breaking rule in the design document.

A5. Configurable operation costs

Substitution, insertion, and deletion costs must be supplied at run time via the command line. If no arguments are provided, all operations should be set to a cost of 1. Portions of our analysis and program extensions rely on the user being able to easily pass in arguments over multiple executions.

Command-line interface. Every submission must support the following command line parameters:

None

```
--input FILE --sub N --ins N --del N --impl {naive|memo|table}
[--verbose] [--counters]
```

Flag	Meaning	Default
<code>--input FILE</code>	Path to a TSV input file	required
<code>--sub N</code>	Substitution cost	1
<code>--ins N</code>	Insertion cost	1
<code>--del N</code>	Deletion cost	1
<code>--impl</code>	Which implementation to run: <code>naive</code> (A1), <code>memo</code> (A2), <code>table</code> (A3)	<code>table</code>
<code>--verbose</code>	Also print the alignment block for each pair	off
<code>--counters</code>	Also emit a counter summary per pair, to counters.txt	off

Costs are **non-negative integers**. If negative numbers or non-integers are entered for costs, the program should tell the user where they failed and terminate. If the required input file is not provided, notify the user and terminate execution. Failure to properly read a correct input file **is your program failing to start**. At your discretion additional flags may be added.

My own test file will be run against your program during grading with this interface, a sample of which is provided on Canvas alongside this document. Depending on implementation our counts may differ somewhat, but the sequence of edit operations should be the same.

Output format

Print one line per input pair in the following format. Your output must be in **input order**

None

```
A<TAB>B<TAB>distance
```

With **--verbose**, print the three-line alignment block immediately after each result line, indented by two spaces:

```
None
kitten      sitting      3
  k i t t e n -
  s i t t i n g
  S . . . S . I
```

With **--counters**, write one line per pair to **counters.txt**, which is created only with this flag

```
None
reads=1234 writes=567 comparisons=890 calls=0 total_operations=2691
check=OK
```

Remember to reset your counter after each pair. **check=OK** or **check=MISMATCH** reflects the return of **Counters.check()**.

When I run your code, nothing else should be printed. Please comment out or remove your debug statements, timing code (which you will need for the empirical study), and similar such datapoints.

A6. Test suite

You must provide the following tests at minimum: empty strings (both, and one of each), identical strings, single-character differences (for each operation), strings sharing no characters, a case with multiple distinct optimal alignments, at least one non-unit cost scheme, and at least three cases you computed by hand with the work shown, with string lengths of at least 5 for both A and B. Your base test suite should be a small separate program or a function that can be called in your main (perhaps via a CLI flag!). Note which it is in your submission.

A7. Instrumentation

Instrument A1, A2, and A3 using the `Counters` file for your respective language. Use the following counting conventions.

Event	Counts as
Reading one DP table or memo cell	1 read (whether the memo hits or misses)
Writing one DP table or memo cell	1 write
Reading one character from an input string	1 read
Testing <code>A[i] == B[j]</code>	2 reads + 1 comparison
Taking the minimum of k values	$k - 1$ comparisons
Base-case initialization of a cell	1 write
Arithmetic (adding a cost to a subproblem value)	not counted
Loop indices, bounds checks, allocation	not counted

`calls` increments once per invocation of the recursive function in A1 and A2. For A3 (tabulated, no recursion) leave `calls` at 0. As a reminder, `calls` is not included in `total_operations`.

Every increment to `reads`, `writes`, or `comparisons` must be accompanied by an increment to `total_operations`. **`check()` must return `True` on every run you report.** If `check()` returns `False`, it will be counted as that pair failing. Be careful to ensure that you capture the true number of operations (naive will consume *many* operations, potentially more than a standard int can contain).

Known environment constraints

- **Python:** the default recursion limit is about 1000, A2 can raise `RecursionError` when `|A| + |B|` approaches that. `sys.setrecursionlimit()` raises the ceiling, but pushing it too high can cause other interesting errors.
- **Java:** will throw `StackOverflowError` when the recursion gets too high. The `-Xss` JVM flag increases thread stack size.

If/when you run into these errors, report the largest input size at which your memoized implementation completes successfully, describe the failure mode, and relate it to your

maximum-recursion-depth measurement from B3. Note that A1 will certainly also cause failure states on large inputs, but it is impractical to try and run it outside of the small test cases.

2. Part B — Empirical study

You will measure and evaluate operation counts and raw execution time for this study.

Operation counts as instrumented above allow us to accurately describe our algorithms. They are deterministic, machine-independent, and reproducible.

Run time describes *this implementation, on this machine, on this day*. It is noisy and it includes everything the language runtime does on your behalf. If I run your code on my machine, I should see the same operations but gather a different run time. If I run the code at a fresh startup vs when I have 50 Google Chrome tabs open, I might expect a difference in performance, etc.

B1. When does Naive become unusable?

Run A1 against A2 on inputs of increasing length. Capture **both** operation counts and run time. Find and report the input size at which the naive version becomes impractical on your machine — define "impractical" yourself and state your threshold (e.g. >60 seconds). Plot both implementations on shared axes.

Report `calls` for both.

B2. Tabulation Asymptotic Complexity

Run A3 on strings of growing length `n`, using `|A| = |B| = n`. Test the following values of `n` [10,20,40,80,160,320,640,1280]. Fill in a table with the following headers: [n, total_ops, ratio, run time (ms), ratio]. Plot `total_operations` against `n` on log-log axes. Do the same for run time (separate graphs of course). When does the ratio between consecutive points stabilize (if it does)

B3. Memoized vs. Tabulated

Compare A2 and A3 on **identical** inputs across all of:

- `total_operations`
- Run time (ms)
- table cells allocated, for A3 this will be the size of the full table; for A2 it will be the number of memo entries actually created (is there a formula to find this, or is it input dependent?)
- **maximum recursion depth** reached for A2

These two implementations compute the same recurrence and will have very similar operation counts. If your implementations are correct, the operation counts should be similar, but what about their run times? The number of cells allocated?

B4. Cost weights

Run each of the following cost schemes against the same input set. Report, for each scheme and each string pair: the distance, and the composition of the optimal edit script (counts of S, I, D, and matches).

Scheme	sub	ins	del
S1	1	1	1
S2	2	1	1
S3	3	1	1
S4	1	1	5

Then answer these three questions in your report.

(a) The substitution threshold. Across S1-S3, substitution should go from common to completely absent. State the general condition on the three costs under which substitution can appear in *some* optimal script, and prove it in a short paragraph. This should be a succinct argument, not a research paper

(b) The LCS identity. Under S3, verify that for every pair in your data,

None

$$\text{distance}(A, B) = |A| + |B| - 2 \cdot \text{LCS}(A, B)$$

Computer LCS using our code from September 17th. Explain why this identity holds when substitution is dominated and fails when it is not.

(c) Symmetry. Under S4, find a concrete pair where $\text{distance}(A, B) \neq \text{distance}(B, A)$. Explain why. Then state the condition on the costs under which edit distance *is* symmetric.

Methodology requirements

Your report must state: machine (CPU, RAM, OS), programming language and version, repetitions per data point and how you aggregated them (mean, median, maximum and minimum, etc), whether warm-up runs were discarded, and how input strings were generated.

B5. Edge Case Analysis

Generate random string pairs of fixed length `n` over alphabets of at least three different sizes — for example 2, 4, and 26 — holding `n` constant. Report `total_operations` for A3 on each, and answer the following question: “Does the runtime of the tabular edit distance algorithm have an asymptotically different best and worst case scenario?”

Input data

Draw on at least two of: English word pairs, short DNA sequences, and randomly generated strings over a controlled alphabet. Whenever you use random strings, **state the alphabet size**.

3. Part C — Design and analysis document

PDF or Markdown. Expected length 5–8 pages excluding plots and appendices.

§1. Problem statement. What an input is, what an output is, what "optimal" means for this problem.

§2. Recurrence and optimal substructure. Your recurrence, with costs as parameters rather than constants. Remember that the relation appears even in the iterative tabular form. Argue the correctness of your algorithm: why does an optimal solution to the full problem contain optimal solutions to its subproblems? State and justify your base cases. State your tie-breaking rule for reconstruction.

§3. Test plan. What you are testing, why those cases, and what a failure of each would tell you. **Include the hand-computed cases with work shown.**

§4. Complexity and instrumentation. The time and space complexity for all three implementations. State what reconstruction costs in both time and space. Confirm that `check()` passed on every reported run, and note any place where the counting convention in §A7 was ambiguous for your code and how you resolved it.

§5. Empirical study. Results from B1–B6: plots with labeled axes and units, methodology statement, cost-weight findings with the three B4 answers, and the B5 investigation.

§6. Extension. See section 4 below.

§7. Source log. A short table summarizing the resources used throughout this project

Source	What I took from it	Where it appears in my work
CLRS 4e §14.4	LCS recurrence structure, adapted for substitution cost	§2, my recurrence
[linked video]	Visual intuition for traceback direction	A4
Claude	Debugged an off-by-one in table initialization	A3

Rules:

- Log AI tool use here.
- No formal citation format needed — a link or chapter number is enough.
- Be specific. "Helped me understand DP" is not a row.
- You are not graded on the number or prestige of your sources, rather this is a way for you to report how effectively the sources aided you.

4. Part D — Extension (choose one)

Implement it, test it, and add a design-document section explaining how the extension changes the relation your algorithm was built on. Your understanding of the change is just as important as the implementation. Instrument your extension with **Counters** as well.

D1. Hirschberg's linear-space alignment. $O(mn)$ time, $O(\min(m,n))$ space, via divide-and-conquer. Demonstrate on strings large enough that the full table is uncomfortable, and use your **cells allocated** measurement from B3 to show the payoff quantitatively.

D2. Affine gap penalties (Gotoh's algorithm). A gap opening cost plus a smaller per-character extension cost, so one long gap beats several short ones. Requires tracking multiple states per cell, describe what those states are and how you track them.

D3. Smith–Waterman local alignment. Best-matching *substring* pair rather than full-string alignment. The recurrence change is small; explain why such a small change has that level of impact.

D4. Damerau–Levenshtein distance. Add transposition of adjacent characters as a fourth operation with its own configurable cost (extend your CLI with `--transpose N`). The restricted and unrestricted variants are different problems, you are only required to implement one.

D5. Banded / Ukkonen's algorithm. If you only need to know whether the distance is at most k , compute only a diagonal band. Add `--max-k N` to your CLI, state the complexity in terms of k , and measure the speedup in both operation counts and run time.

5. Deliverables and grading

Submission checklist

A single archive containing:

- Source code for A. Code must be executable.
- Extension code (Part D)
- Design and analysis document (Part C)
- Plots from Part B, embedded or in a `figures/` folder.
- Raw measurement data (CSV) used to produce every plot
- Any input files you generated
- `README`: how to build, how to run, how to run tests, which extension you chose, known defects. Include how to run your test suite (A6)

Rubric (100 points)

Component	Points
Core algorithm correctness (A1–A3, A5, required CLI) and test suite (A6)	20
Reconstruction — edit script and printed alignment (A4)	9
Instrumentation (A7) — convention applied consistently, <code>check()</code> passes	5
Experiments B1–B3 — run correctly, plots legible, methodology stated	10
B4 cost-weight study — four schemes, threshold proof, LCS identity, symmetry	9
B6 — alphabet-size measurements and the input-insensitivity explanation	5
Design doc §2 — recurrence and optimal-substructure argument	12
Design doc §3 — test plan and hand-computed cases	5
Design doc §4 — complexity analysis with justification	6
Design doc §7 — source log	4

Extension (Part D) — implementation and recurrence explanation	10
Code styling and README completeness	5
Total	100

6. Input file formats

Two file types, one is for verification of correctness and the other contains your larger dataset(s) for the Empirical Study (B). Lines beginning with **#** and blank lines are ignored — your parser must skip them. Strings will not contain tab characters.

Costs are never stored in data files. They come from the command line, so the same data file runs under all four B4 schemes without duplication.

Verification files: ***_verification.tsv**

Three columns: string A, string B, expected distance **under unit costs**. Your code should ignore empty lines and comment lines.

None

```
# A <TAB> B <TAB> expected_distance (unit costs)
kitten      sitting      3
flaw lawn 2
intention execution 5
```

A sample file in this format is on Canvas; as mentioned above the “correctness” of your algorithms will be verified by a file in this format. The costs are there for verification that your algorithm generated the right distance

Empirical Study — ***_study.tsv**

Two columns, no expected value (these will be run with multiple different costs). These files may be large. In your report describe the alphabet for each file.

None

```
# A <TAB> B
ACGTACGTTGCA      ACGTTGCAACGT
```

Generate your own input files for B1–B3 and submit them with your archive. It is possible your dataset may make Canvas unhappy, if that is the case include a permalink to the .tsv file (host it on your google drive) inside your submission. For large randomly generated datasets, submit the generator script and its seed instead of the data itself. We want a fixed seed for consistency in the operation count.

7. Resources

You are not expected to read all of these. Pick what you need and log what you use. This list is also not exhaustive. If you find helpful resources elsewhere, be sure to log them so I can include them in next year's assignment.

Start here

- **CLRS 4e §14.4** (LCS) and **§14.3** (elements of dynamic programming).
- **Jeff Erickson, *Algorithms*, Chapter 3** — free at <https://jeffe.cs.illinois.edu/teaching/algorithms/>.
- **MIT OpenCourseWare 6.006**, dynamic programming lectures — <https://ocw.mit.edu/>.
- **VisuAlgo** (<https://visualgo.net/>) and the **USF visualizations** (<https://www.cs.usfca.edu/~galles/visualization/>) for watching a table fill when the traceback direction is not clicking.

Origins of the Algorithm in Research

- **V. I. Levenshtein (1966)**, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady* 10(8):707–710.
- **R. A. Wagner and M. J. Fischer (1974)**, "The string-to-string correction problem," *Journal of the ACM* 21(1):168–173. The algorithm you are implementing, in its original presentation.
- **S. B. Needleman and C. D. Wunsch (1970)**, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of Molecular Biology* 48(3):443–453.
- **T. F. Smith and M. S. Waterman (1981)**, "Identification of common molecular subsequences," *Journal of Molecular Biology* 147(1):195–197. Under three pages. Relevant to D3.
- **D. S. Hirschberg (1975)**, "A linear space algorithm for computing maximal common subsequences," *Communications of the ACM* 18(6):341–343. Relevant to D1.
- **O. Gotoh (1982)**, "An improved algorithm for matching biological sequences," *Journal of Molecular Biology* 162(3):705–708. Relevant to D2.

A more modern analysis

- **A. Backurs and P. Indyk (2015)**, "Edit Distance Cannot Be Computed in Strongly Subquadratic Time (unless SETH is false)," *STOC 2015*, pp. 51–58, <https://arxiv.org/abs/1412.0348>. Journal version: *SIAM J. Comput.* 47(3):1087–1097, 2018.

Data and practice

- **Rosalind** — <https://rosalind.info/> bioinformatics problems with real sequence data.
- **LeetCode 72** (Edit Distance)